Session 01:

Ngoại lệ trong Java

Rikkei Academy®
where the dream begins

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

NỘI DUNG

1. GIỚI THIỆU NGOẠI LỆ
2. PHÂN LOẠI NGOẠI LỆ
3. THROW & THROWS
4. TRY CATCH FINALLY

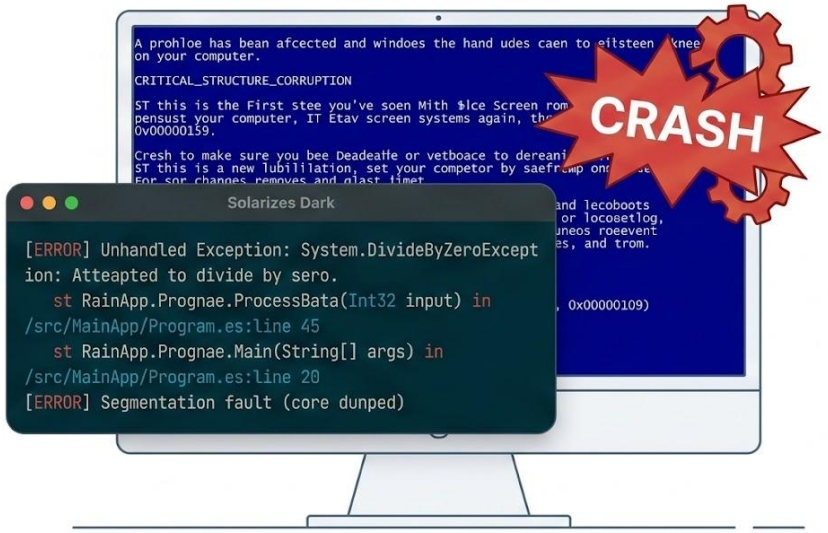
© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

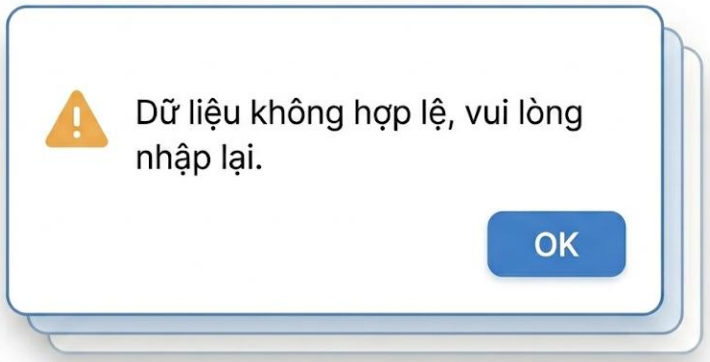
1

GIỚI THIỆU NGOẠI LỆ

Tại sao phải xử lý ngoại lệ?



Không xử lý



Có xử lý ngoại lệ

Vấn đề: Nhập sai dữ liệu (chữ vào ô số) hoặc chia cho 0.

Hậu quả: App dừng đột ngột, mất dữ liệu, treo tài nguyên.

Giải pháp: Chỉ gửi cảnh báo lỗi mà không làm dừng hệ thống

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

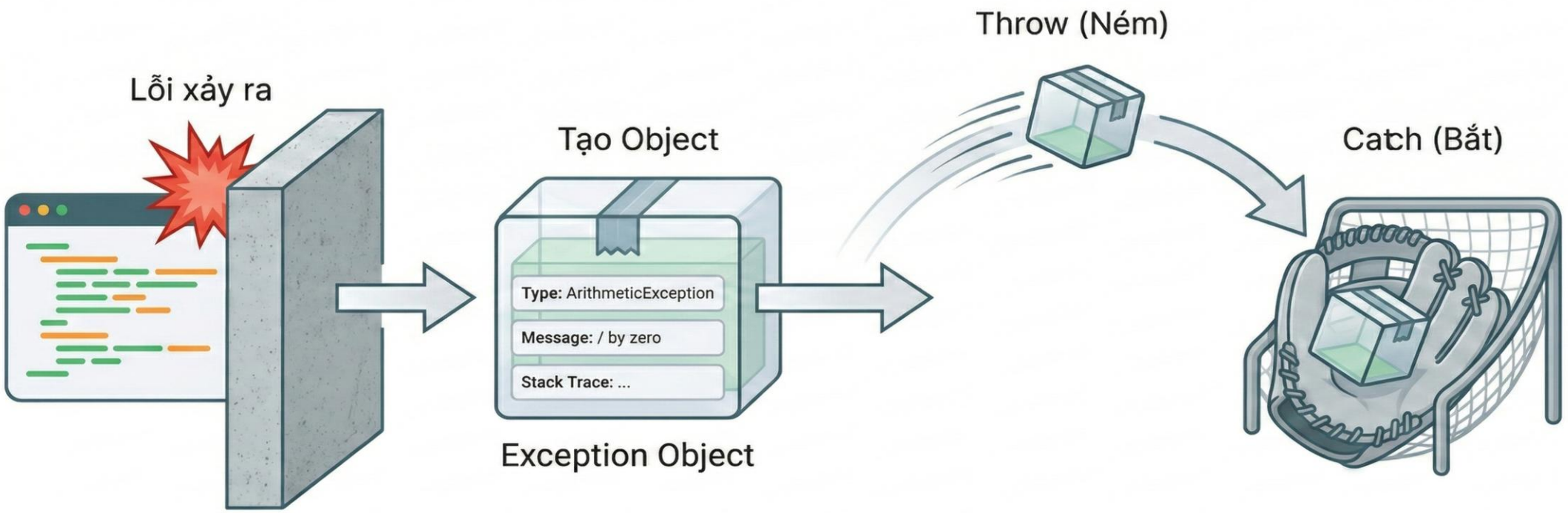
Khái niệm Exception

Định nghĩa: Sự kiện bất thường xảy ra tại Runtime, phá vỡ luồng thực thi (Normal Flow).

Trong Java, **Exception** là một **Object**, không chỉ là một mã lỗi.

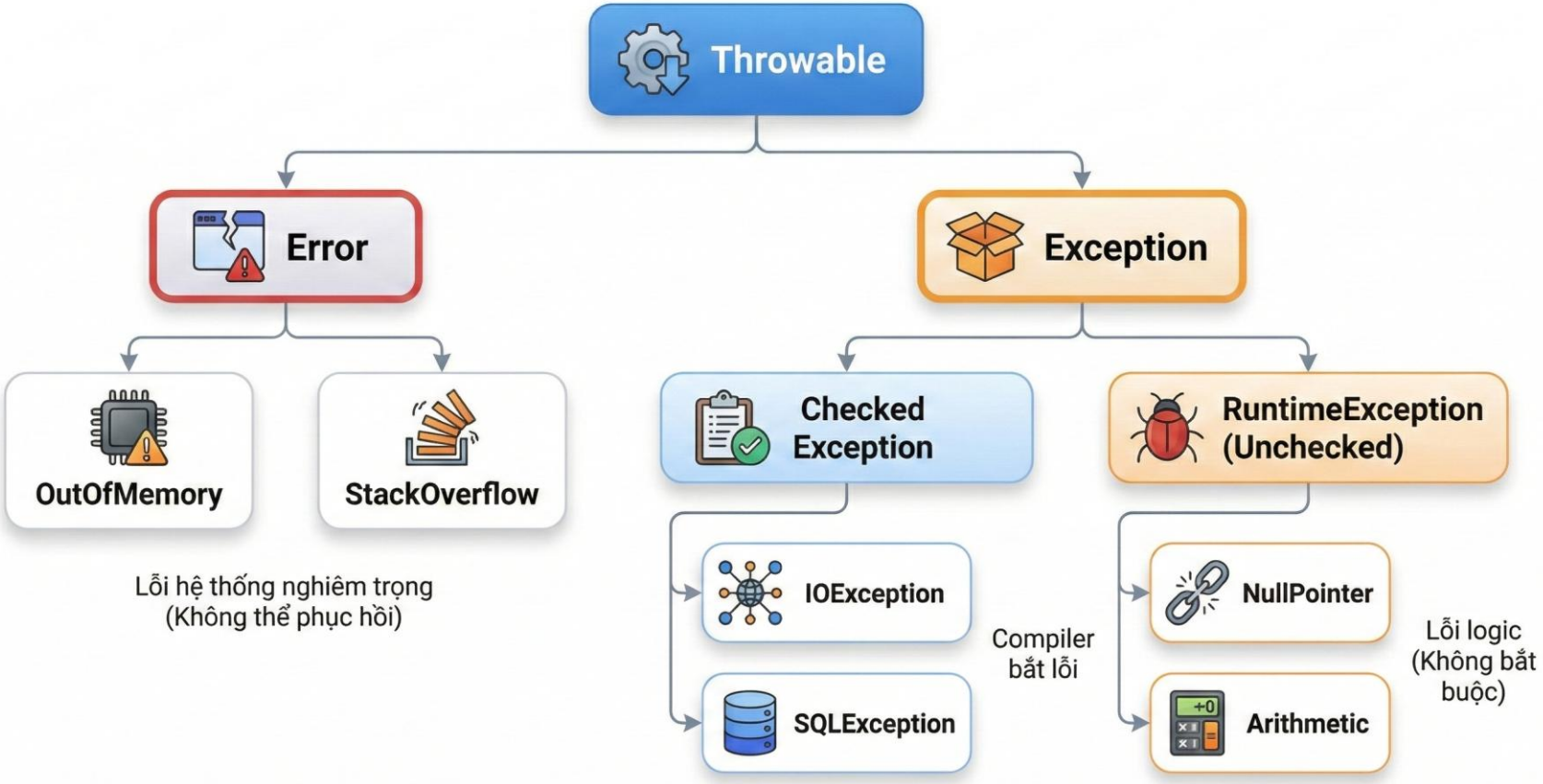
© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Bản chất của Exception



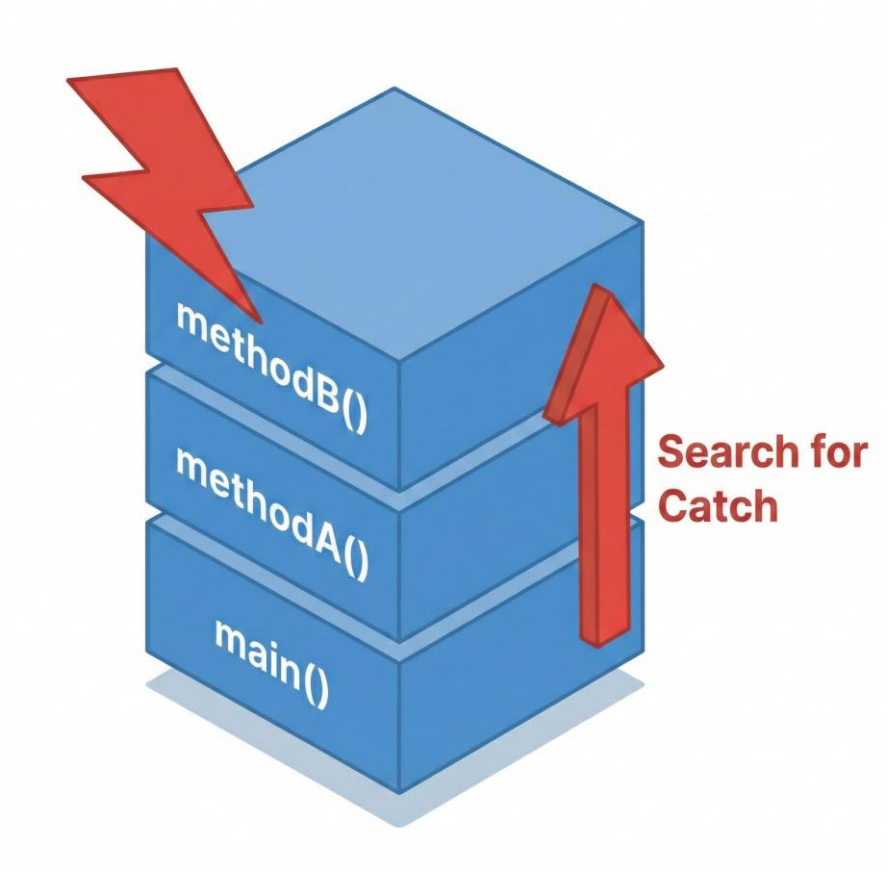
© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Sơ đồ phân cấp Throwable



© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Cơ chế call stack & stack trace



1. Gặp lỗi: JVM tạo Object Exception tại dòng lệnh lỗi.

2. Pause: Tạm dừng luồng thực thi hiện tại.

3. Search: Tìm ngược lên trên CallStack.

4. Result:

- Tìm thấy catch -> Xử lý và chạy tiếp.
- Không thấy -> Crash chương trình + In Stack Trace.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Mục tiêu của Exception Handling

- ✓ Ứng dụng không bị dừng (Crash) đột ngột
- ✓ Bảo vệ dữ liệu và tài nguyên đang xử lý
- ✓ Thông báo lỗi thân thiện, rõ ràng cho người dùng
- ✓ Cho phép ứng dụng tiếp tục hoạt động hoặc dừng an toàn (graceful shutdown)
- ✓ Hỗ trợ debug và bảo trì dễ dàng hơn

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

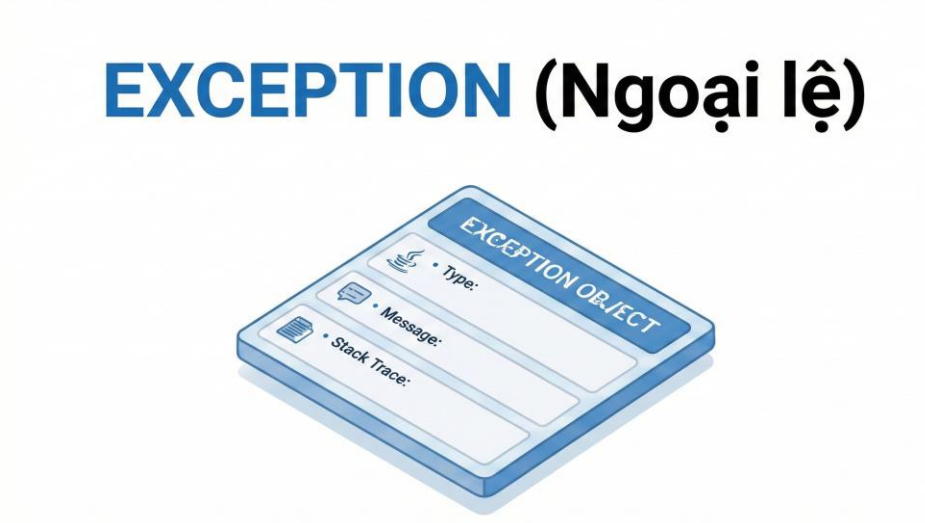
RIKKEI Academy®
where the dream begins

2

PHÂN LOẠI NGOẠI LỆ

Phân biệt Error với Exception

- Sự cố nghiêm trọng, không thể khôi phục.
- Thường do môi trường hệ thống.
- VD: OutOfMemoryError, StackOverflowError



- Các điều kiện bất thường có thể xảy ra trong chương trình.
- Có thể được xử lý (catch) và khôi phục.
- VD: NullPointerException, IOException

Tóm lại: Error là sự cố nghiêm trọng, Exception là các trường hợp có thể quản lý được.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

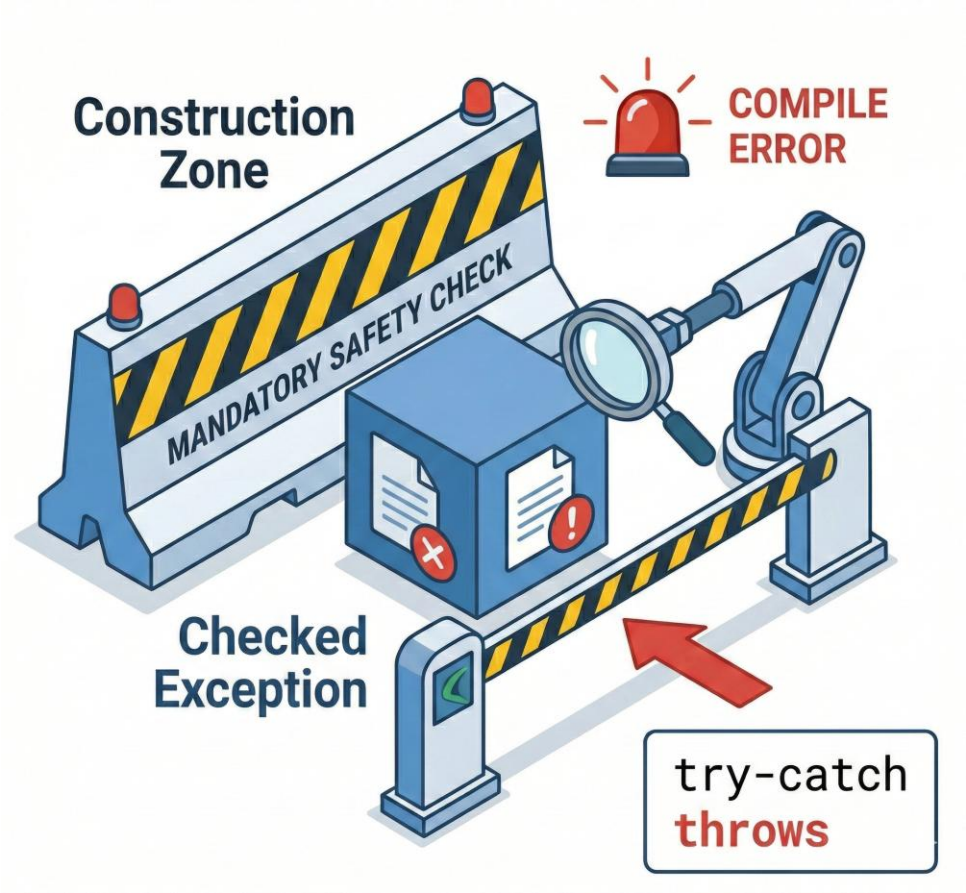
CHECKED VS UNCHECKED

- **Kịch bản:** Đọc file data.txt
- **Rủi ro** : File bị xóa, ổ cứng hỏng (**Khách quan**).
- **Compiler**: **Bắt buộc** phải dự phòng.

- **Kịch bản:** Tính toán / Truy cập mảng
- **Rủi ro**: Lỗi logic của lập trình viên (**Chủ quan**).
- **Compiler**: Lỗi của anh, anh **tự sửa logic**.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

NGOẠI LỆ CÓ KIỂM TRA (CHECKED EXCEPTIONS)



Đặc điểm: Kế thừa từ Exception (trừRuntimeException).

Bản chất: Lỗi ngoài tầm kiểm soát (Môitrường, I/O, Network).

Quy tắc biên dịch:

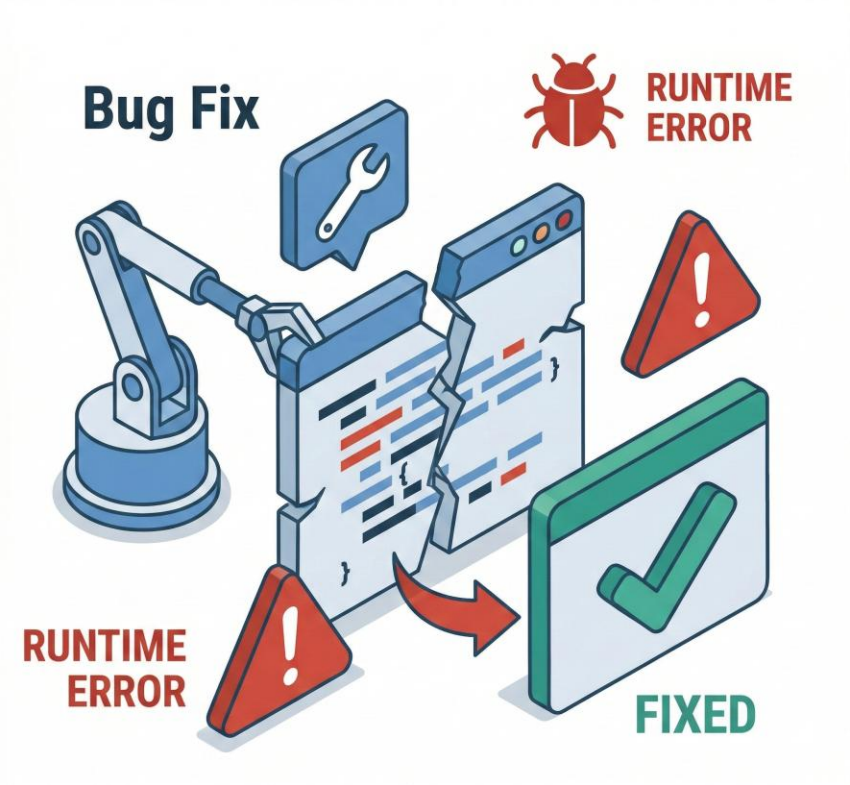
- **BẮT BUỘC** phải xử lý (try-catch hoặc throws).
- Nếu quên -> Code không thể Compile(Lỗi biên dịch).

Ví dụ:

- IOException
- SQLException
- ClassNotFoundException

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

NGOẠI LỆ KHÔNG CÓ KIỂM TRA (UNCHECKED EXCEPTIONS)



Đặc điểm: Kế thừa từ RuntimeException.

Bản chất: Lỗi do logic lập trình kém **(Programming Error)**

Quy tắc: Compiler KHÔNG bắt buộc xử lý.

Best Practice - Tư duy phòng ngừa:

- Đừng dùng try-catch để giấu lỗi logic.
- Hãy dùng if-else để kiểm tra (DefensiveProgramming).

Code tệ:

```
try {
    print(obj.toString())
} catch...
```

Code chuẩn:

```
if (obj != null) {
    print(obj.toString())
}
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Khi nào cần try-catch

Xử lý **Checked Exceptions** bắt buộc (VD: IO, SQL)

Ngăn chặn ứng dụng bị **Crash** do Unchecked Exceptions

Cung cấp thông báo lỗi **thân thiện** cho người dùng

Dọn dẹp tài nguyên quan trọng (finally, try-with-resources)

Ghi log lỗi chi tiết để debug và bảo trì

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Vùng nguy hiểm: Bao bọc đoạn mã có nguy cơ lỗi.

Lưới an toàn: Bắt lỗi, ngăn crash

```java
try {
    // Vùng nguy hiểm
    int result = 10 / 0;
} catch (ArithmeticException e) {
    // Lưới an toàn
    System.out.println("Không thể chia cho 0!");
}
```

Điểm gây lỗi

Kết quả: Thay vì dừng lại, chương trình in thông báo và tiếp tục chạy.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Khi nào cần validate dữ liệu thay vì bắt lỗi

Validate Dữ Liệu

- **Chủ động** kiểm tra trước khi xử lý (User input, API calls).
- **Ngăn chặn** dữ liệu không hợp lệ, đảm bảo tính toàn vẹn.
- **Phản hồi** lỗi rõ rang, thân thiện cho người dùng.
- **Ví dụ:** Email sai định dạng, trường bắt buộc bị trống.

Bắt Lỗi

- **Bị động** xử lý khi sự cố xảy ra (Unexpected runtime errors).
- **Phục hồi** từ trạng thái lỗi, tránh crash chương trình.
- **Ghi log** lỗi chi tiết, thông báo lỗi hệ thống.
- **Ví dụ:** Lỗi kết nối DB, file bị xóa bất ngờ.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

2

THROW & THROWS

Phân biệt throw và throws

throw (Hành động)

- **Vị trí:** Trong thân hàm.
- **Tác dụng:** Kích hoạt một ngoại lệ cụ thể.

```
throw new Exception("Hết tiền!");
```

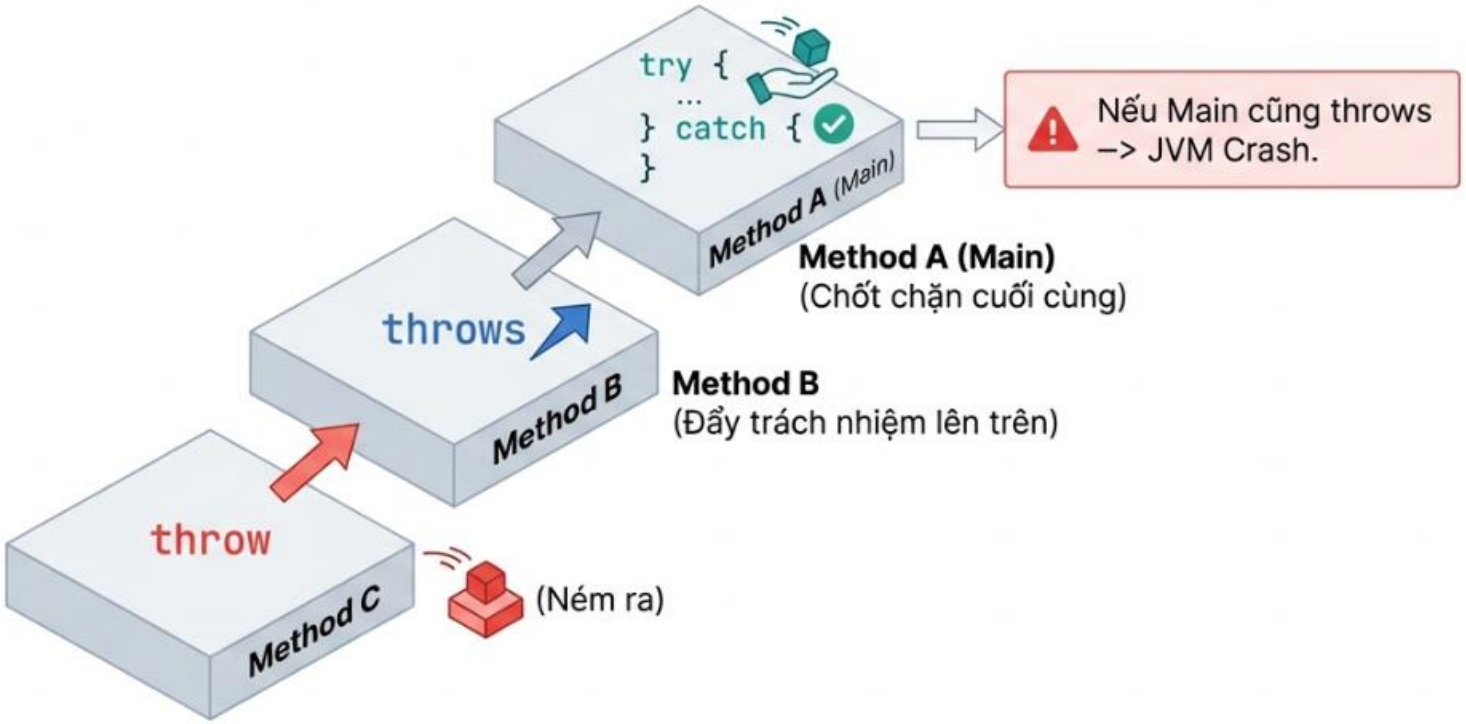
throws (Khai báo)

- **Vị trí:** Sau tên hàm (Method Signature).
- **Tác dụng:** Cảnh báo người gọi hàm.

```
public void readFile() throws IOException
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Sự lan truyền & trách nhiệm (propagation)



Quy tắc: Xử lý (catch) ở nơi có đủ thông tin để quyết định (ví dụ: UI Layer).

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Khi nào nên tự ném Exception

Chủ động kiểm tra và báo lỗi để bảo vệ logic nghiệp vụ.

- **Kiểm tra tham số đầu vào** (InputValidation) không hợp lệ.
  - Ví dụ: tuổi < 0, giá tiền âm.
- **Vi phạm quy tắc nghiệp vụ** (BusinessRule Violation)
  - Ví dụ: 'Số dư không đủ'.
- **Bao bọc** (Wrap) các ngoại lệ cấp thấp thành ngoại lệ có ý nghĩa hơn.
  - Ví dụ: SQLException -> DatabaseAccessException.

Ví dụ Code:

```
public void setAge(int age) {
  if (age < 0) {
    throw new IllegalArgumentException(
          'Tuổi không thể âm!');
  }
  this.age = age;
}
```

Lợi ích: Mã nguồn rõ ràng, dễ debug, ngăn chặn lỗi lan truyền sai lệch.

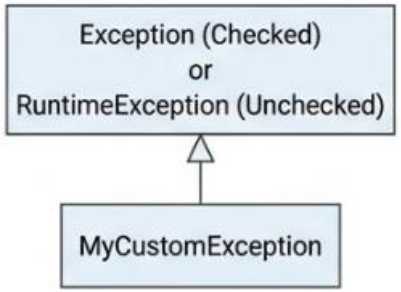
© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Custom Exception (ngoại lệ tùy chỉnh)

Khái niệm và lợi ích

- **Định nghĩa:** Ngoại lệ do lập trình viên tự tạo, kế thừa từ Exception/Runtime Exception.
- **Lợi ích:** Tăng tính rõ ràng, mô tả lỗi cụ thể (VD: InvalidAgeException).
- Dễ dàng phân loại và xử lý lỗi riêng biệt.

Cách tạo và sử dụng




```java
class MyCustomException extends Exception {
    public MyCustomException(String msg) { super(msg); }
}

// Sử dụng:
if (age < 0) {
    throw new MyCustomException("Tuổi không hợp lệ!");
}
```

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

Anti-pattern: throw new Exception()

VẤN ĐỀ (THE PROBLEM)

```
Throw new Exception("Lỗi rồi!");
```

- Mất thông tin loại lỗi (Loss of specific error type).
- Buộc người gọi phải **catch** (Exception) chung chung.
- Khó xử lý lỗi cụ thể, dễ bỏ sót lỗi khác.

GIẢI PHÁP ĐÚNG (THE SOLUTION)

```
throw new IllegalArgumentException("Tuổi không hợp lệ: " + age);
```

- Dùng **Exception cụ thể** (Use specific Exceptions).
- Rõ ràng, dễ **bảo trì**, dễ xử lý cho người gọi.
- Tạo **Custom Exception** nếu cần thiết.

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

4. TRY CATCH FINALLY

CẤU TRÚC TRY - CATCH - FINALLY

```java
try {
    // Cố gắng parse dữ liệu người dùng nhập
    int birthYear = Integer.parseInt(input);
    System.out.println("Tuổi của bạn là: " + (2026 - birthYear));
    // Giả sử có lệnh return ở đây, khối finally VẪN SẼ CHẠY
    // return;
} catch (NumberFormatException e) {
    // Bắt chính xác loại lỗi NumberFormatException
    System.out.println("Lỗi: Bạn phải nhập một con số hợp lệ, không phải chữ!");
    // In ra dấu vết lỗi cho lập trình viên (thường dùng khi debug)
    // e.printStackTrace();
} finally {
    // Khối này LUÔN CHẠY dù có lỗi hay không
    System.out.println("Thực hiện dọn dẹp tài nguyên trong finally...");
    scanner.close(); // Đóng Scanner để tránh rò rỉ bộ nhớ (memory leak)
}
```

Chứa code rủi ro, dừng lại ngay khi lỗi

Lưới đỡ an toàn, chỉ chạy khi có lỗi

Luôn luôn chạy dọn dẹp hiện trường

- **Block try:** Thực thi logic chính.
- **Block catch:** Xử lý hậu quả.
- **Block finally:** Giải phóng tài nguyên (Scanner, File, DB).

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

TỔNG KẾT

- ❑ Hiểu và nắm được khái niệm về ngoại lệ (Exception)
- ❑ Phân biệt rõ ràng giữa Error và Exception, Checked Exception (bắt buộc xử lý) và Unchecked Exception (lỗi logic).
- ❑ Biết cách sử dụng từ khóa throw
- ❑ Thành thạo cấu trúc try-catch-finally
- ❑ Biết cách tạo và sử dụng Custom Exception

© 2022 By Rikkei Academy - Rikkei Education - All rights reserved.

RIKEI Academy®
where the dream begins

KẾT THÚC

HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN